

Formatting Ranges

Document #: P2286R2
Date: 2021-07-19
Project: Programming Language C++
Audience: LEWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

[1 Revision History](#)

[2 Introduction](#)

[2.1 Implementation Experience](#)

[3 Proposal Considerations](#)

[3.1 What types to print?](#)

[3.2 What representation?](#)

[3.3 What additional functionality?](#)

[3.4 What about format specifiers?](#)

[3.5 How to support those views which are not const-iterable?](#)

[3.6 Specifying formatters for ranges](#)

[3.7 format or std::cout?](#)

[3.8 What about vector<bool>?](#)

[4 Proposal](#)

[4.1 Wording](#)

[4.1.1 Concept formattable](#)

[4.1.2 Additional formatter specializations in <format>](#)

[4.1.3 The join formatter](#)

[4.1.4 Formatter for pair](#)

[4.1.5 Formatter for tuple](#)

[4.1.6 Formatter for vector<bool>::reference](#)

[5 References](#)

§ 1 Revision History

Since [[P2286R1](#)], adding a sketch of wording.

[[P2286R0](#)] suggested making all the formatting implementation-defined. Several people reached out to me suggesting in no uncertain terms that this is unacceptable. This revision lays out options for such formatting.

§ 2 Introduction

[LWG3478] addresses the issue of what happens when you split a string and the last character in the string is the delimiter that you are splitting on. One of the things I wanted to look at in research in that issue is: what do *other* languages do here?

For most languages, this is a pretty easy proposition. Do the split, print the results. This is usually only a few lines of code.

Python:

```
print("xyx".split("x"))
```

outputs

```
['', 'y', '']
```

Java (where the obvious thing prints something useless, but there's a non-obvious thing that is useful):

```
import java.util.Arrays;

class Main {
    public static void main(String args[]) {
        System.out.println("xyx".split("x"));
        System.out.println(Arrays.toString("xyx".split("x")));
    }
}
```

outputs

```
[Ljava.lang.String;@76ed5528
[, y]
```

Rust (a couple options, including also another false friend):

```
use itertools::Itertools;

fn main() {
    println!("{}", "xyx".split('x'));
    println!("{}", "xyx".split('x').format(", "));
    println!("{}", "xyx".split('x').collect::<Vec<_>>());
}
```

outputs

```
Split(SplitInternal { start: 0, end: 3, matcher: CharSearcher { haystack:
    "xyx", finger: 0, finger_back: 3, needle: 'x', utf8_size: 1,
    utf8_encoded: [120, 0, 0, 0] }, allow_trailing_empty: true,
    finished: false })
[, y, ]
["", "y", ""]
```

Kotlin:

```
fun main() {
    println("xyx".split("x"));
}
```

```
}
```

outputs

```
[, y, ]
```

Go:

```
package main
import "fmt"
import "strings"

func main() {
    fmt.Println(strings.Split("xyx", "x"));
}
```

outputs

```
[ y ]
```

JavaScript:

```
console.log('xyx'.split('x'))
```

outputs

```
[ '', 'y', '' ]
```

And so on and so forth. What we see across these languages is that printing the result of split is pretty easy. In most cases, whatever the print mechanism is just works and does something meaningful. In other cases, printing gave me something other than what I wanted but some other easy, provided mechanism for doing so.

Now let's consider C++.

```
#include <iostream>
#include <string>
#include <ranges>
#include <format>

int main() {
    // need to predeclare this because we can't split an rvalue string
    std::string s = "xyx";
    auto parts = s | std::views::split('x');

    // nope
    std::cout << parts;

    // nope (assuming std::print from P2093)
    std::print("{} ", parts);

    std::cout << "[";
    char const* delim = " ";
```

```

for (auto part : parts) {
    std::cout << delim;

    // still nope
    std::cout << part;

    // also nope
    std::print("{} ", part);

    // this finally works
    std::ranges::copy(part, std::ostream_iterator<char>(std::cout));

    // as does this
    for (char c : part) {
        std::cout << c;
    }
    delim = ", ";
}
std::cout << "]\n";
}

```

This took me more time to write than any of the solutions in any of the other languages. Including the Go solution, which contains 100% of all the lines of Go I've written in my life.

Printing is a fairly fundamental and universal mechanism to see what's going on in your program. In the context of ranges, it's probably the most useful way to see and understand what the various range adapters actually do. But none of these things provides an `operator<<` (for `std::cout`) or a formatter specialization (for `format`). And the further problem is that as a user, I can't even do anything about this. I can't just provide an `operator<<` in `namespace std` or a very broad specialization of `formatter` - none of these are program-defined types, so it's just asking for clashes once you start dealing with bigger programs.

The only mechanisms I have at my disposal to print something like this is either

1. nested loops with hand-written delimiter handling (which are tedious and a bad solution), or
2. at least replace the inner-most loop with a `ranges::copy` into an output iterator (which is more differently bad), or
3. Write my own formatting library that I *am* allowed to specialize (which is not only bad but also ridiculous)
4. Use `fmt::format`.

§ 2.1 Implementation Experience

That's right, there's a fourth option for C++ that I haven't shown yet, and that's this:

```

#include <ranges>
#include <string>
#include <fmt/ranges.h>

int main() {

```

```
std::string s = "xyx";
auto parts = s | std::views::split('x');

fmt::print("{}\n", parts);
fmt::print("[{}]\n", fmt::join(parts, ","));
}
```

outputting

```
{}, {'y'}
[{},'y']
```

And this is great! It's a single, easy line of code to just print arbitrary ranges (include ranges of ranges).

And, if I want to do something more involved, there's also `fmt::join`, which lets me specify both a format specifier and a delimiter. For instance:

```
std::vector<uint8_t> mac = {0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
fmt::print("{:02x}\n", fmt::join(mac, " "));
```

outputs

```
aa:bb:cc:dd:ee:ff
```

`fmt::format` (and `fmt::print`) solves my problem completely. `std::format` does not, and it should.

§ 3 Proposal Considerations

The Ranges Plan for C++23 [[P2214R0](#)] listed as one of its top priorities for C++23 as the ability to format all views. Let's go through the issues we need to address in order to get this functionality.

§ 3.1 What types to print?

The standard library is the only library that can provide formatting support for standard library types and other broad classes of types like ranges. In addition to ranges (both the concrete containers like `vector<T>` and the range adaptors like `views::split`), there are several very commonly used types that are currently not printable.

The most common and important such types are `pair` and `tuple` (which ties back into Ranges even more closely once we adopt `views::zip` and `views::enumerate`). `fmt` currently supports printing such types as well:

```
fmt::print("{}\n", std::pair(1, 2));
```

outputs

```
(1, 2)
```

Another common and important set of types are `std::optional<T>` and `std::variant<Ts...>`. `fmt` does not support printing any of the sum types. There is not an obvious representation for them in C++ as

there might be in other languages (e.g. in Rust, an `Option<i32>` prints as either `Some(42)` or `None`, which is also the same syntax used to construct them).

However, the point here isn't necessarily to produce the best possible representation (users who have very specific formatting needs will need to write custom code anyway), but rather to provide something useful. And it'd be useful to print these types as well. However, given that `optional` and `variant` are both less closely related to `Ranges` than `pair` and `tuple` and also have less obvious representation, they are less important.

§ 3.2 What representation?

There are several questions to ask about what the representation should be for printing.

1. Should `vector<int>{1, 2, 3}` be printed as `{1, 2, 3}` (as `fmt` currently does and as the type is constructed in C++) or `[1, 2, 3]` (as is typical representationally outside of C++)?
2. Should `pair<int, int>{3, 4}` be printed as `{3, 4}` (as the type is constructed) or as `(3, 4)` (as `fmt` currently does and is typical representationally outside of C++)?
3. Should `char` and `string` in the context of ranges and tuples be printed as quoted (as `fmt` currently does) or unquoted (as these types are typically formatted)?

What I'm proposing is the following:

```
std::vector<int> v = {1, 2, 3};
std::map<std::string, char> m = {"hello", 'h'}, {"world", 'w'};

std::print("v={}. m={}\n", v, m);
```

print:

```
v=[1, 2, 3]. m=[("hello", 'h'), ("world", 'w')]
```

That is: ranges are surrounded with `[]`s and delimited with `", "`. Pairs and tuples are surrounded with `()`s and delimited with `", "`. Types like `char`, `string`, and `string_view` are printed quoted.

It is more important to me that ranges and tuples are visually distinct (in this case `[]` vs `()`, but the way that `fmt` currently does it as `{}` vs `()` is also okay) than it would be to quote the string-y types. My rank order of the possible options for the map `m` above is:

Ranges	Tuples	Quoted?	Formatted Result
<code>[]</code>	<code>()</code>	✓	<code>[("hello", 'h'), ("world", 'w')]</code>
<code>{}</code>	<code>()</code>	✓	<code>{("hello", 'h'), ("world", 'w')}</code>
<code>[]</code>	<code>()</code>	✗	<code>[(hello, h), (world, w)]</code>
<code>{}</code>	<code>()</code>	✗	<code>{(hello, h), (world, w)}</code>
<code>{}</code>	<code>{}</code>	✗	<code>{{hello, h}, {world, w}}</code>

My preference for avoiding `{}` in the formatting is largely because it's unlikely the results here can be used directly for copying and pasting directly into initialization anyway, so the priority is simply having

visual distinction for the various cases.

With quoting, the question is how does the library choose if something is string-like and thus needs to be quoted. Currently, `fmtlib` makes this determination by checking for the presence of certain operations (`p->length()`, `p->find('a')`, and `p->data()`). We could instead add a variable template called `enable_formatting_as_string` to allow for opting into this kind of formatting. This approach isn't perfect, since we easily end up with situations like:

```
vector<string> words = {" some ", " words ", "here"};
std::print("{}\n", words); // [" some ", " words ", "here"]

auto trimmed = words | transform(views::trim_whitespace);
std::print("{}\n", trimmed); // [['s', 'o', 'm', 'e'], ['w', 'o', 'r', 'd', 's'], ['h', 'e', 'r', 'e']]
```

You probably want the trimmed strings to still print quoted as `"some"` rather than `['s', 'o', 'm', 'e']`, and having a variable template wouldn't solve that unless you apply it to `trim_whitespace_view` as well. This suggests maybe going one step further and introduced some kind of `views::quoted` that exists solely to indicate that a particular range of char should be formatted as quoted:

```
auto trimmed2 = words | transform(views::trim_whitespace | views::quoted);
std::print("{}\n", trimmed2); // ["some", "words", "here"]
```

But that's not necessarily sufficient for handling cases like wanting to print a `tuple<R, int>` where `R` is some range of char that you want to print quoted - there's no "view" on a tuple that we can easily apply. So rather than worry necessary about solving all possible formatting problems, this paper simply proposes `enable_formatting_as_string` (which will be `true` for specializations of `basic_string` and `basic_string_view`), which can be easily applied to the myriad of other string-like types out there, on the basis of being good enough.

§ 3.3 What additional functionality?

There's three layers of potential functionality:

1. Top-level printing of ranges: this is `fmt::print!("{}", r)`;
2. A format-joiner which allows providing a format specifier for each element and a delimiter: this is `fmt::print("{}:02x", fmt::join(r, ":"))`.
3. A more involved version of a format-joiner which takes a delimiter and a callback that gets invoked on each element. `fmt` does not provide such a mechanism, though the Rust `itertools` library does:

```
let matrix = [[1., 2., 3.],
               [4., 5., 6.]];
let matrix_formatter = matrix.iter().format_with("\n", |row, f| {
    f(&row.iter().format_with(", ", |elt, g|
        g(&elt)))
    });
assert_eq!(format!("{}", matrix_formatter),
           "1, 2, 3\n4, 5, 6");
```

This paper suggests the first two and encourages research into the third.

§ 3.4 What about format specifiers?

The implementation experience in `fmt` is that directly formatting ranges does *not* support any format specifiers, but `fmt::join` supports providing a specifier per element as well as providing the delimiter and wrapping brackets.

We could add the same format specifier support for direct formatting of ranges as `fmt::join` supports, but it doesn't seem especially worthwhile. If you don't care about formatting, `"{}"` is all you need. If you do care about formatting, it's likely that you care about more than just the formatting of each individual element — you probably care about other things do. At which point, you'd likely need to use `fmt::join` anyway.

That seems like the right mix of functionality to me.

§ 3.5 How to support those views which are not `const`-iterable?

There are several C++20 range adapters which are not `const`-iterable. These include `views::filter` and `views::drop_while`. But `std::format` takes its arguments by Args `const&...`, so how could they be printable?

`fmt` handles this just fine even with its formatter specialization taking by `const` by converting the range [to a view first](#).

So we can do something like:

```
template <range R, typename Char>
struct formatter<R, Char>
{
    template <typename FormatContext>
    auto format(R const& rng, FormatContext& ctx) {
        auto values = std::views::all(rng);
        // ...
    }
};
```

But, this still doesn't cover all the cases. If `R` is a type such that `R const` is a view but `R` is move-only, that won't compile. We can work around this case. But if `R` is a type such that `view<R>` and `not range<R const>` and `not copyable<R>`, there's really no way of getting this to work without changing the API.

Notably, the proposed `std::generator<T>` is one such type [\[P2168R0\]](#)

If we do want to support this case (and `fmt` does not), then we will need to change the API of `format` to take by forwarding reference instead of by `const` reference. That is, replace

```
template<class... Args>
string format(string_view fmt, const Args&... args);
```


with:

```
template<class... Args>
string format(string_view fmt, Args&&... args);
```

But it's not that easy. While we would need `std::generator<T>` to bind to `Args&&` rather than `Args const&`, there are other cases which have the exact opposite problem: bit-fields need to bind to `Args const&` and cannot bind to `Args&&`.

So we can't really change the API to support this use-case without breaking other use-cases. But there are workarounds for this case:

- `fmt::join`, which does take by forwarding reference
- `ranges::ref_view` can wrap an lvalue of such a view, the resulting view would be `const`-iterable.

For example:

```
auto ints_coro(int n) -> std::generator<int> {
    for (int i = 0; i < n; ++i) {
        co_yield i;
    }
}

fmt::print("{} ", ints_coro(10)); // error
fmt::print("[{}]", fmt::join(ints_coro(10), ", ")); // ok

auto v = ints_coro(10);
fmt::print("{} ", std::ranges::ref_view(v)); // ok, if tedious

fmt::print("{} ", ints_coro(10) | ranges::to<std::vector>); // ok, but
expensive
fmt::print("{} ", views::iota(0, 10)); // ok, but harder to implement
```

You can see an example of formatting a move-only, non-`const`-iterable range on compiler explorer [[fmt-non-const](#)].

§ 3.6 Specifying formatters for ranges

It's quite important that a `std::string` whose value is `"hello"` gets printed as `hello` rather than something like `[h, e, l, l, o]`.

This would basically fall out no matter how we approach implementing such a thing, so in of itself is not much of a concern. However, for users who have either custom containers or want to customize formatting of a standard container for their own types, they need to make sure that they can provide a specialization which is more constrained than the standard library's for ranges. To ensure that they can do that, I think we need to be clear about the specific constraint we use when we specify this, and thus this paper proposes a user-facing concept `formattable` that other parts of this proposal will directly use.

§ 3.7 `format` or `std::cout`?

Just format is sufficient.

§ 3.8 What about `vector<bool>`?

Nobody expected this section.

The `value_type` of this range is `bool`, which is formattable. But the reference type of this range is `vector<bool>::reference`, which is not. In order to make the whole type formattable, we can either make `vector<bool>::reference` formattable (and thus, in general, a range is formattable if its reference types is formattable) or allow formatting to fall back to constructing a value for each reference (and thus, in general, a range is formattable if either its reference type or its `value_type` is formattable).

For most ranges, the `value_type` is `remove_cvref_t<reference>`, so there's no distinction here between the two options. And even for `zip`, there's still not much distinction since it just wraps this question in tuple since again for most ranges the types will be something like `tuple<T, U>` vs `tuple<T&, U const&>`, so again there isn't much distinction.

`vector<bool>` is one of the very few ranges in which the two types are truly quite different. So it doesn't offer much in the way of a good example here, since `bool` is cheaply constructible from `vector<bool>::reference`. Though it's also very cheap to provide a formatter specialization for `vector<bool>::reference`.

Rather than having the library provide a default fallback that lifts all the reference types to `value_types`, which may be arbitrarily expensive for unknown ranges, this paper proposes a format specialization for `vector<bool>::reference`. Or, rather, since it's actually defined as `vector<bool, Alloc>::reference`, this isn't necessarily feasible, so instead this paper proposes a specialization for `vector<bool, Alloc>` at top level.

§ 4 Proposal

The standard library should add specializations of formatter for:

- any type `T` such that `T const` satisfies range and whose reference is formattable,
- `pair<T, U>` if `T` and `U` are formattable,
- `tuple<Ts...>` if all of `Ts...` are formattable,
- `vector<bool, Alloc>::reference` (which formats as a `bool`).

Ranges should be formatted as `[x, y, z]` while tuples should be formatted as `(a, b, c)`. `std::array` is tuple-like, but not a tuple, it's treated as a range. In the context of formatting ranges, pairs, and tuples, character types (in the 6.8.2 [\[basic.fundamental\]](#) sense) or string-like (e.g. `string`, `string_view`, controlled by `enable_formatting_as_string`) should be formatted as being quoted (characters using `'` and strings using `"`).

Formatting ranges does not support any additional format specifiers.

The standard library should also add a utility `std::format_join` (or any other suitable name, knowing that `std::views::join` already exists), following in the footsteps of `fmt::join`, which allows the user to provide more customization in how ranges and tuples get formatted.

For types like `std::generator<T>` (which are move-only, non-const-iterable ranges), users will have to either use `std::format_join` facility or use something like `ranges::ref_view` as shown earlier.

§ 4.1 Wording

The wording here is grouped by functionality added rather than linearly going through the standard text.

§ 4.1.1 Concept `formattable`

First, we need to define a user-facing concept. We need this because we need to constrain `formatter` specializations on whether the underlying elements of the `pair/tuple/range` are `formattable`, and users would need to do the same kind of thing for their types. This is tricky since formatting involves so many different types, so this concept will never be perfect, so instead we're trying to be good enough:

Change 20.20.1 [[format.syn](#)]:

```
namespace std {
    // ...

    // [format.formatter], formatter
    template<class T, class charT = char> struct formatter;

    // [format.parse.ctx], class template basic_format_parse_context
    template<class charT> class basic_format_parse_context;
    using format_parse_context = basic_format_parse_context<char>;
    using wformat_parse_context = basic_format_parse_context<wchar_t>;

+ // [format.formattable], formattable
+ template<class T, class charT>
+     concept formattable = see below;

    // ...
}
```

Add a clause `[format.formattable]` under 20.20.5 [[format.formatter](#)] and likely after 20.20.5.1 [[formatter.requirements](#)]

- 1 Let `fmt-iter-for<charT>` be an implementation-defined type that models `output_iterator<const charT>` ([`iterator.concept.output`]).

```
template<class T, class charT>
concept formattable-impl =
    semiregular<formatter<T, charT>> &&
    requires (formatter<T, charT> f,
              const T t,
              basic_format_context<fmt-iter-for<charT>, charT> fc,
              basic_format_parse_context<charT> pc) {
```

```

        { f.parse(pc) } ->
        same_as<basic_format_parse_context<charT>::iterator>;
        { f.format(t, fc) } -> same_as<fmt-iter-for<charT>>;
    };

template<class T, class charT>
concept formattable = formattable-impl<remove_cvref_t<T>, charT>;

```

- 2 A type T and a character type charT model formattable if $\text{formatter}<T, \text{charT}>$ meets the *Formatter* requirements ([`formatter.requirements`]).

§ 4.1.2 Additional formatter specializations in `<format>`

Change 20.20.1 [[format.syn](#)]:

```

namespace std {
    // ...

    // [format.formatter], formatter
+ template<class T>
+     inline constexpr bool enable_formatting_as_string = see below;

    template<class T, class charT = char> struct formatter;

+ // [format.range], range formatter
+ template<class R, class charT>
+     concept default-formattable-range = // exposition only
+         ranges::input_range<const R> &&
+             formattable<ranges::range_reference_t<const R>, charT>
+         || ranges::input_range<R> && ranges::view<R> && copyable<R> &&
+             formattable<ranges::range_reference_t<R>, charT>
+
+ template<class charT, default-formattable-range<charT>, R>
+     struct formatter<R, charT>;

    // ...
}

```

Add to... somewhere:

```

template<class T>
    inline constexpr bool enable_formatting_as_string = false;

template<class charT, class traits, class Allocator>
    inline constexpr bool enable_formatting_as_string<basic_string<charT,
        traits, Allocator>> = true;

template<class charT, class traits>
    inline constexpr bool enable_formatting_as_string<basic_string_view<charT,
        traits>> = true;

```

- * *Remarks:* Pursuant to [`namespace.std`], users may specialize `enable_formatting_as_string` to `true` for any cv-unqualified program-defined type T which models *default-formattable-range*< charT >. [*Note:* Users may do so to ensure that

a program-defined string type formats as "hello" rather than as ['h', 'e', 'l', 'l', 'o'] -end note].

Add the new clause [format.range]:

```
template<class charT>
inline constexpr auto format-maybe-quote = // exposition only
    []<class T>(const T& t){
        if constexpr (is_same_v<T, charT>) {
            return format(STATICALLY-WIDEN<charT>("{}'"), t);
        } else if constexpr (enable_formatting_as_string<T>) {
            return format(STATICALLY-WIDEN<charT>("{}\{}"), t);
        } else {
            return format(STATICALLY-WIDEN<charT>("{}"), t);
        }
    };
```

```
template<class charT, default-formattable-range<charT>, R>
struct formatter<R, charT> {
    template <class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

    template <class FormatContext>
        typename FormatContext::iterator
            format(const R& range, FormatContext& ctx);
};
```

```
template <class ParseContext>
constexpr typename ParseContext::iterator
    parse(ParseContext& ctx);
```

- 1 Let T denote the type `ranges::range_reference_t<const R>` if `const R` models `ranges::range` and `ranges::range_reference_t<R>` otherwise.
- 2 *Throws:* `format_error` if `ctx` does not refer to an empty *format-spec* or anything that `formatter<remove_cvref_t<T>, charT>().parse(ctx)` throws.
- 3 *Returns:* `ctx.begin()`.

```
template <class FormatContext>
typename FormatContext::iterator
    format(const R& range, FormatContext& ctx);
```

- 4 *Effects:* Let `r` be `range` if `const R` models `ranges::range` and `views::all(range)` otherwise. Writes the following into `ctx.out()`:

- (4.1) — `STATICALLY-WIDEN<charT>("[")`
- (4.2) — for each element, `e`, of the range `r`:
 - (4.2.1) — `format-maybe-quote<charT>(e)`
 - (4.2.2) — `STATICALLY-WIDEN<charT>(", ")`, unless `e` is the last element of `r`

— *STATICALLY-WIDEN*`<charT>("]")`

Returns: an iterator past the end of the output range

Returns: an iterator past the end of the output range

§ 4.1.3 The join formatter

Change 20.20.1 [[format.syn](#)]:

```
namespace std {
    // ...

    // [format.error], class format_error
    class format_error;

+ // [format.join], a join formatter
+ template <ranges::input_range V, class charT>
+     requires ranges::view<V> &&
+         formattable<ranges::range_reference_t<V>, charT>
+ class format-join-impl; // exposition only
+
+ template <ranges::input_range V, class charT>
+     requires ranges::view<V> &&
+         formattable<ranges::range_reference_t<V>, charT>
+ struct formatter<format-join-impl<V, charT>, charT>;
+
+
+ template <ranges::input_range R>
+     requires formattable<ranges::range_reference_t<R>, char>
+ constexpr format-join-impl<ranges::ref_view<remove_reference_t<R>>, char>
+     format_join(R&& range, string_view sep);
+
+ template <ranges::input_range R>
+     requires formattable<ranges::range_reference_t<R>, wchar_t>
+ constexpr format-join-impl<ranges::ref_view<remove_reference_t<R>>,
+         wchar_t>
+     format_join(R&& range, wstring_view sep);
}
```

Add a new clause [format.join]:

1 The function template `format_join` is a convenient utility to provide a custom *format-spec* to apply to each element when formatting a range, along with a custom delimiter.

2 [Example:

```
cout << format("{:02x}", format_join(vector{10,20,30,40,50,60}, ":")); //  
prints 0a:14:1e:28:32:3c
```

```
-end example]
```

```
template <ranges::input_range V, class charT>
    requires ranges::view<V> &&
               formattable<ranges::range_reference_t<V>, charT>
class format-join-view { // exposition only
    V view;                // exposition only
```

```

    basic_string_view<charT> sep; // exposition only

public:
    constexpr format-join-view(V v, basic_string_view<charT> s);
};

constexpr format-join-view(V v, basic_string_view<charT> s)

```

3 *Effects*: Direct-non-list-initializes *view* with `std::move(v)` and *sep* with *s*.

```

template <ranges::input_range V, class charT>
    requires ranges::view<V> &&
           formattable<ranges::range_reference_t<V>, charT>
class formatter<format-join-impl<V, charT>, charT> {
    formatter<remove_cvref_t<ranges::range_reference_t<V>>, charT> fmt; //
        exposition only

public:
    template <typename ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

    template <typename FormatContext>
        typename FormatContext::iterator
            format(const format-join-impl<V, charT>& j, FormatContext& ctx);
};

template <typename ParseContext>
    constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

```

4 *Effects*: Equivalent to `return fmt.parse(ctx);`

```

template <typename FormatContext>
    typename FormatContext::iterator
        format(const format-join-impl<V, charT>& j, FormatContext& ctx);

```

5 *Effects*: Write the following into `ctx.out()`:

- (3.1) — For each element *e* of *j.view*:
- (3.1.1) — `fmt.format(e, ctx)`
- (3.1.2) — *j.sep*, unless *e* is the last element of *j.view*

6 *Returns*: an iterator past the end of the output range

```

template <ranges::input_range R>
    requires formattable<ranges::range_reference_t<R>, char>
constexpr format-join-view<ranges::ref_view<remove_reference_t<R>>, char>
    format_join(R&& range, string_view sep);

template <ranges::input_range R>
    requires formattable<ranges::range_reference_t<R>, wchar_t>
constexpr format-join-view<ranges::ref_view<remove_reference_t<R>>, wchar_t>
    format_join(R&& range, wstring_view sep);

```

7 *Effects:* Equivalent to `return {ranges::ref_view(range), sep};`

§ 4.1.4 Formatter for pair

Add to 20.2.1 [\[utility.syn\]](#)

```
namespace std {
    // ...

    // [pairs], class template pair
    template<class T1, class T2>
        struct pair;

+   template<class charT, formattable<charT> T1, formattable<charT> T2>
+       struct formatter<pair<T1, T2>, charT>;
+
+   template <formattable<char> T1, formattable<char> T2>
+       constexpr see below format_join(const pair<T1, T2>& p, string_view sep);
+
+   template <formattable<wchar_t> T1, formattable<wchar_t> T2>
+       constexpr see below format_join(const pair<T1, T2>& p, wstring_view
+           sep);

    // ...
};
```

Add a new subclass `[pair.format]` under 20.4 [\[pairs\]](#)

```
template<class charT, formattable<charT> T1, formattable<charT> T2>
    struct formatter<pair<T1, T2>, charT> {
        template <typename ParseContext>
            constexpr typename ParseContext::iterator
                parse(ParseContext& ctx);

        template <typename FormatContext>
            typename FormatContext::iterator
                format(const pair<T1, T2>& p, FormatContext& ctx);
    };

template <typename ParseContext>
    constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);
```

1 *Throws:* `format_error` if `ctx` does not refer to an empty *format-spec* or anything that `formatter<remove_cvref_t<T1>, charT>().parse(ctx)` or `formatter<remove_cvref_t<T2>, charT>().parse(ctx)` throws.

2 *Returns:* `ctx.begin()`.

```
template <typename FormatContext>
    typename FormatContext::iterator
        format(const pair<T1, T2>& p, FormatContext& ctx);
```

3 *Effects:* Writes the following into `ctx.out()`:

- (3.1) — `STATICALLY-WIDEN<charT>("(")`
 - (3.2) — `format-maybe-quote<charT>(p.first)`
 - (3.3) — `STATICALLY-WIDEN<charT>(", ")`
 - (3.4) — `format-maybe-quote<charT>(p.second)`
 - (3.5) — `STATICALLY-WIDEN<charT>(")")`
- 4 *Returns:* an iterator past the end of the output range

§ 4.1.5 Formatter for tuple

Add to 20.5.2 [\[tuple.syn\]](#)

```
#include <compare>                                // see [compare.syn]

namespace std {
    // [tuple.tuple], class template tuple
    template<class... Types>
        class tuple;

+   template<class charT, formattable<charT>... Types>
+       struct formatter<tuple<Types...>, charT>;
+
+   template <formattable<char>... Types>
+       constexpr see below format_join(const tuple<Types...>& t, string_view
+           sep);
+
+   template <formattable<wchar_t>... Types>
+       constexpr see below format_join(const tuple<Types...>& t, wstring_view
+           sep);

    // ...
};
```

Add a new subclause [\[tuple.format\]](#) under 20.5 [\[tuple\]](#)

```
template<class charT, formattable<charT>... Types>
    struct formatter<tuple<Types...>, charT> {
        template <typename ParseContext>
            constexpr typename ParseContext::iterator
                parse(ParseContext& ctx);

        template <typename FormatContext>
            typename FormatContext::iterator
                format(const tuple<Types...>& t, FormatContext& ctx);
    };

template <typename ParseContext>
    constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);
```

- 1 *Throws:* `format_error` if `ctx` does not refer to an empty *format-spec* or anything that `formatter<remove_cvref_t<T>, charT>().parse(ctx)` throws for each type `T` in

Types....

2 *Returns:* `ctx.begin()`.

```
template <typename FormatContext>
    typename FormatContext::iterator
        format(const tuple<Types...>& t, FormatContext& ctx);
```

3 *Effects:* Writes the following into `ctx.out()`:

(3.1) — *STATICALLY-WIDEN*`<charT>(" ")`

(3.2) — For each element *e* in the tuple *t*:

(3.2.1) — *format-maybe-quote*`<charT>(e)`

(3.2.2) — *STATICALLY-WIDEN*`<charT>(", ")`, unless *e* is the last element of *t*

(3.3) — *STATICALLY-WIDEN*`<charT>(" ")`

4 *Returns:* an iterator past the end of the output range

§ 4.1.6 Formatter for `vector<bool>::reference`

Add to 22.3.6 [\[vector.syn\]](#)

```
namespace std {
    // [vector], class template vector
    template<class T, class Allocator = allocator<T>> class vector;

    // ...

    // [vector.bool], class vector<bool>
    template<class Allocator> class vector<bool, Allocator>;

+ template<class R>
+     inline constexpr bool is-vector-bool-reference = see below; //
+     exposition only

+ template<class R, class charT> requires is-vector-bool-reference<R>
+     struct formatter<R, charT>;
```

Add to [\[vector.bool\]](#) at the end:

```
template<class R>
    inline constexpr bool is-vector-bool-reference = see below;
```

8 The variable template *is-vector-bool-reference*`<T>` is `true` if *T* denotes the type `vector<bool, Alloc>::reference` for some type *Alloc* if `vector<bool, Alloc>` is not a program-defined specialization.

```
template<class R, class charT> requires is-vector-bool-reference<R>
    class formatter<R, charT> {
        formatter<bool, charT> fmt;        // exposition only
```

```

public:
    template <class ParseContext>
        constexpr typename ParseContext::iterator
            parse(ParseContext& ctx);

    template <class FormatContext>
        typename FormatContext::iterator
            format(const R& ref, FormatContext& ctx);
};

template <class ParseContext>
    constexpr typename ParseContext::iterator
        parse(ParseContext& ctx);

```

9 *Effects:* Equivalent to `return fmt.parse(ctx);`

```

template <class FormatContext>
    typename FormatContext::iterator
        format(const R& ref, FormatContext& ctx);

```

10 *Effects:* Equivalent to `return fmt.format(ref, ctx);`

§ 5 References

[fmt-non-const] Barry Revzin. 2021. Formatting a move-only non-const-iterable range.

<https://godbolt.org/z/149YqW>

[LWG3478] Barry Revzin. `views::split` drops trailing empty range.

<https://wg21.link/lwg3478>

[P2168R0] Corentin Jabot, Lewis Baker. 2020-05-16. `generator`: A Synchronous Coroutine Generator Compatible With Ranges.

<https://wg21.link/p2168r0>

[P2214R0] Barry Revzin, Conor Hoekstra, Tim Song. 2020-10-15. A Plan for C++23 Ranges.

<https://wg21.link/p2214r0>

[P2286R0] Barry Revzin. 2021-01-15. Formatting Ranges.

<https://wg21.link/p2286r0>

[P2286R1] Barry Revzin. 2021-02-19. Formatting Ranges.

<https://wg21.link/p2286r1>